Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Algorithm Complexity and Search

Prof. Reza Skandari

Today

- **Search** algorithms
- How to analyse **algorithm complexity**
  - "How does my code slow as my data grows?"

Guessing game

A friend is thinking of a number between 1-100 and you have to guess it.

Whenever you make a guess, the friend tells you whether it is correct, too high, or too low.

How many guesses will you need?

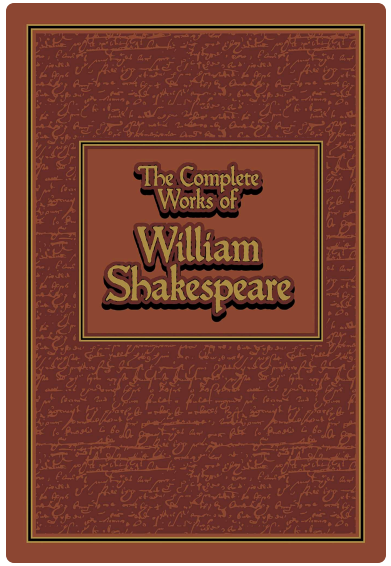
Poll: how many guesses?

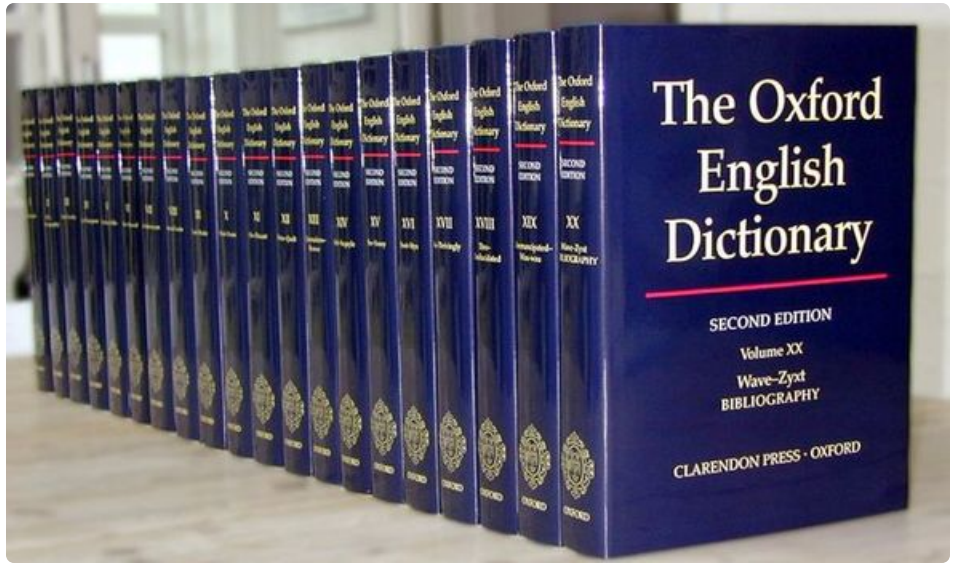
Poll time! Open menti.com and tell us how many guesses you'd need.



Search algorithms

Search for the word "swagger"?





What is the worst we could do?

If the answers were "yes" and "no"?

- Linear search - worst case: go through everything

If the answers are "too low" and "too high"?

- Each time, discard half of remaining numbers
- Binary search - worst case?

Logarithms

Exponentials: $2^4 = 2 \times 2 \times 2 \times 2 = 16$

Logarithm flips the exponential:

- $\log_2 16 = 4$
- "How many 2s do we multiply to get 16?"
- "How many times do we divide 16 by 2 to get to 1?"

For numbers up to 128, we need to make at most $\log_2 128 = 7$ guesses

- What about for numbers up to 1024? 2048? $1\,000\,000$?

Guess the number from 1 to *n*:

- Linear search: at most n guesses
- Binary search: at most $\log_2(n)$ guesses

What do computers do?

1. Perform simple calculations
    - Arithmetic operations, comparisons, assignments, accessing memory
2. Remember the results

Efficiency of an algorithm:

- How much **time** will our computation take?
- How much **memory** will it need?

Linear search

Is `x` in list `A`?

```
def linear_search(A, x):
    for elem in A:
        if elem == x:
            return True
    return False
```

Efficiency:

- How much **time** will our computation take?
- How much **memory** will it need?

How much time will it take?

Simple way: run and time it! But running time depends on:

1. Speed of computer
2. Specifics of implementation
3. Value of input

We can avoid 1 and 2 by measuring **work** (complexity):

- Computers are good at tirelessly repeating "simple" operations
- **These operations are**: arithmetic operations, comparisons, assignments, accessing memory
- Work = **the number of these operations** your algorithm requires

Value of input

Searching for an item in a list?

```
def linear_search(A, x):
    # A is a list of length n
    for elem in A:
        if elem == x:
            return True
    return False
```

Measure number of steps depending on:

- Size of the input (a long list vs. a short list; a list of `n` items)
- `x` could be the first element of `A`
- `x` could not be in `A`

How to give a general complexity measure?

Complexity cases

Cases for a given input size (length of `A`):

- **Best case** — minimum time
- **Worst case** — maximum time
- **Average case** — average or expected time over all possible inputs

We will focus on worst-case analysis:

- Upper bound on running time
- Bonus: usually easier to analyze

Example

```
def sum_up_to(n):
    result = 0                  # 1 step
    while n > 0:                 # 1 step, n times
        result = result + n      # 2 steps, n times
        n = n - 1                 # 2 steps, n times
    return result                 # 1 step
```

Total: $5n + 2$ steps

- As n gets large, 2 is irrelevant
- Arguably, so is 5
  - It's the size of the problem that matters

Example: ignoring constants

Total: $5n + 2$ steps

We will ignore constant factors and lower-order terms:

- They depend on computer and program implementation
- They do not matter for large inputs
- Simplifies comparisons

Another example

```
def f(x):                       # x is positive integer
    ans = 0                     # 1 step
    for i in range(100):        # 2 steps, 100 times
        ans += 1                 # 2 steps, 100 times
    for i in range(x):          # 2 steps, x times
        ans += 1                 # 2 steps, x times
    for i in range(x):          # 2 steps, x times
        for j in range(x):       # 2 steps, x^2 times
            ans -= 1             # 2 steps, x^2 times
    return ans                   # 1 step for return
```

Steps: $402 + 6x + 4x^2$

- x small $\rightarrow$ first loop dominates ($x = 3$)
- x large $\rightarrow$ last loop dominates ($x = 10^6$)
- Only need to consider last (nested) loop for large x
- Does the 4 in $4x^2$ matter? For large x, order of growth much more important

Summary (asymptotic analysis)

1. Measure number of basic operations as a function of input size
2. Focus on worst-case analysis
3. Ignore constant factors and lower-order terms
4. Only care about large inputs
    - Only large problems are interesting
    - What happens when size gets very large?

Formal way to describe this approach:

- Big-O notation: upper bound on worst-case running time

Big-O: bound on runtime growth rate

Let $T(n)$ be the number of steps taken for input size n:

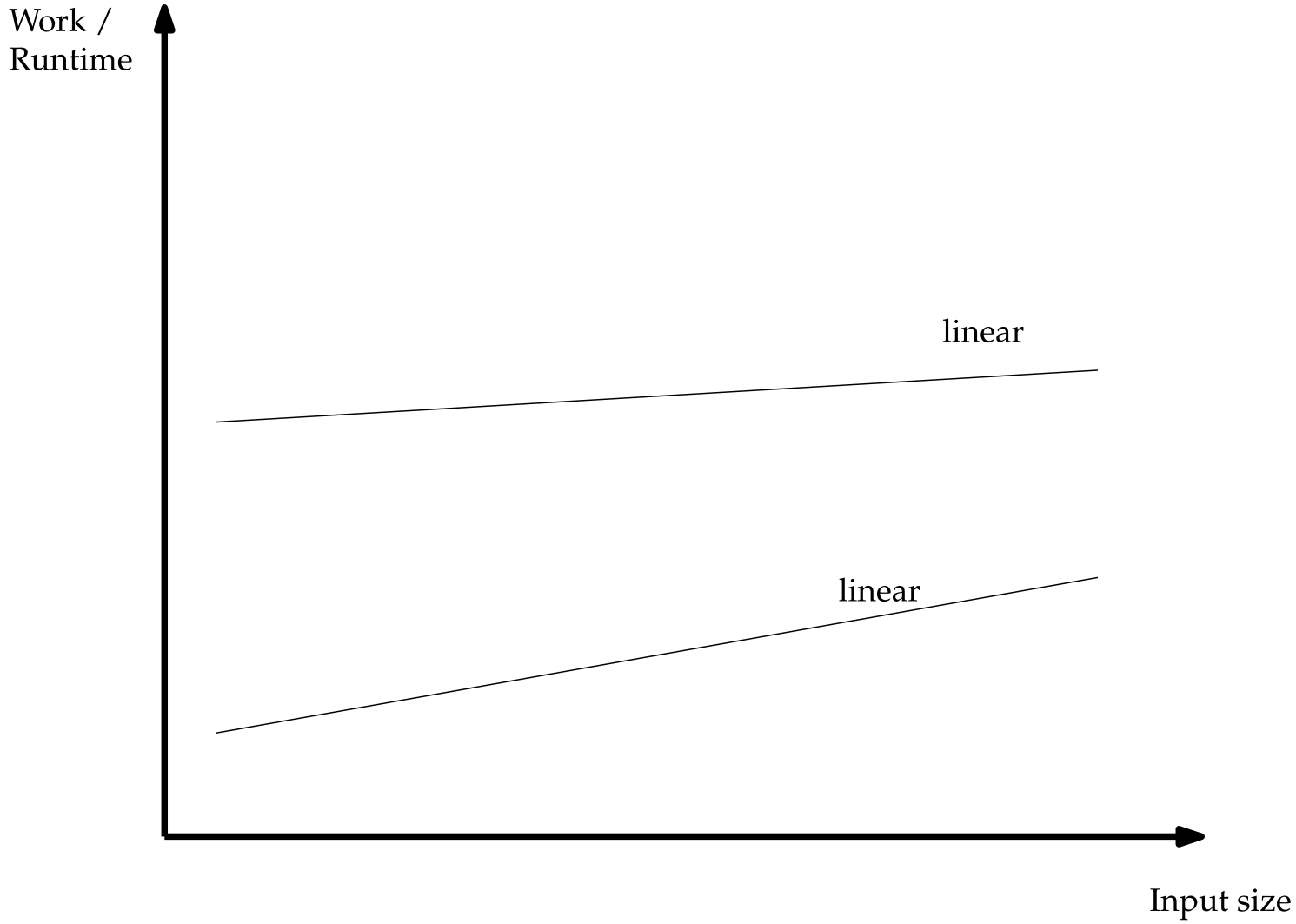
- Example: $T(n) = 402 + 6n + 4n^2$

Definition: $T(n)$ is $O(f(n))$ if for all sufficiently large n, $T(n)$ is bounded above by a constant multiple of $f(n)$

Example: $T(n)$ is $O(n^2)$ if for all large enough n, $T(n)$ is bounded above by a constant multiple of $f(n) = n^2$

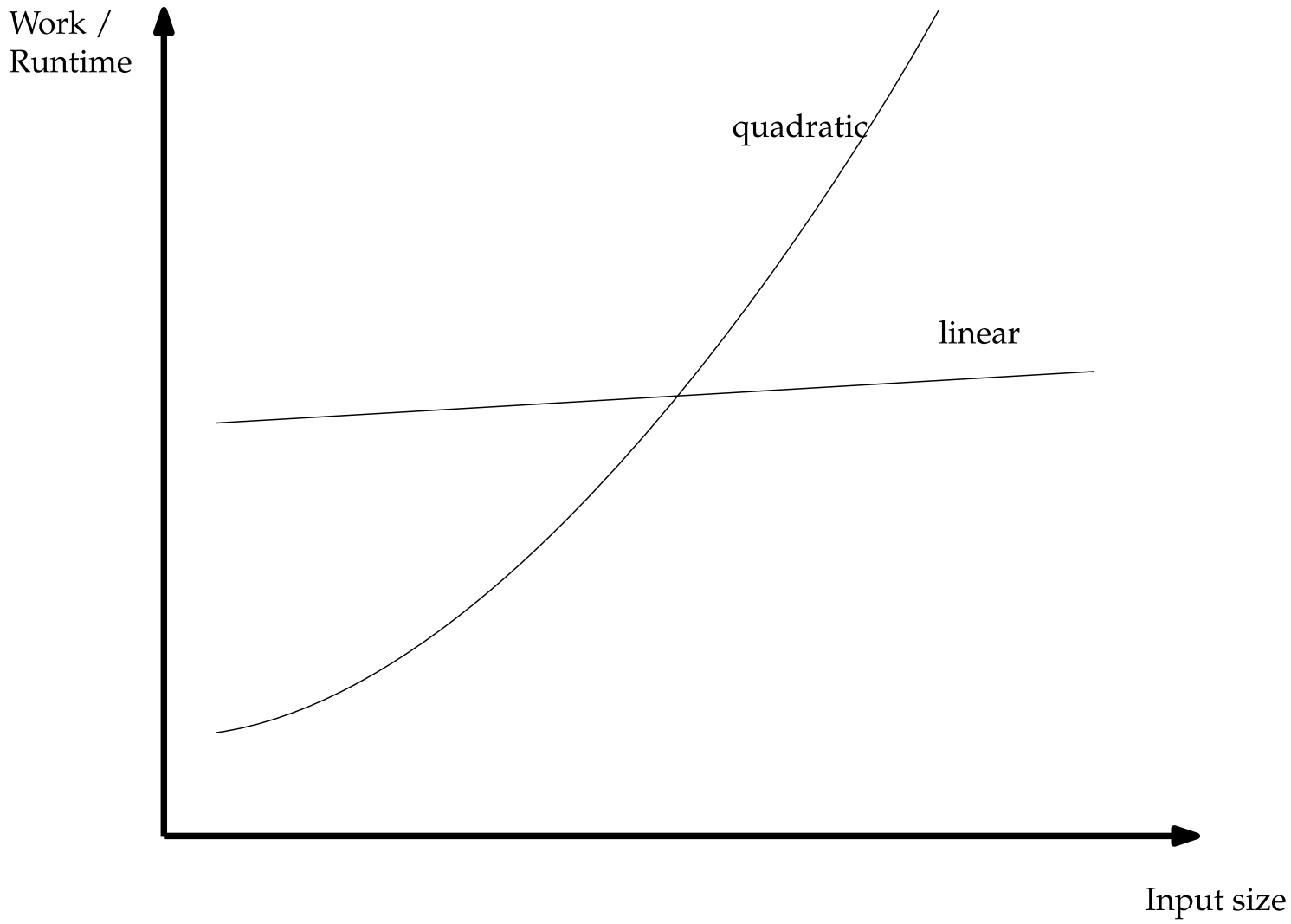
- Gist: for high values of n, does $f(n)$ "grow at least as quickly"?
  - $cf(n) = cn^2$ (for any constant c)
  - $T(n) = 402 + 6n + 4n^2$
- But we cannot say that $T(n)$ is $O(n)$

Growth rates: constants don't matter



Different constants, same growth rate — both are still **linear**, $O(n)$.

Growth rates



Big O tells us how fast the algorithm is

Fast algorithm: worst-case running time grows slowly with input size

- $O(1)$: constant running time — primitive operations
- $O(\log n)$: logarithmic running time
- $O(n)$: linear running time — linear search
- $O(n \log n)$: log-linear time
- $O(n^c)$: polynomial running time
- $O(c^n)$: exponential running time
- $O(n!)$: factorial running time

Test your instincts

Poll time! Open menti.com and vote on the complexity of each snippet.

What is the complexity?

```
def fun_function(n):
    val = 0
    for i in range(n):
        val += 1
    for i in range(1000):
        val += 6
    return val
```

A. Constant time - $O(1)$

B. Linear time - $O(n)$

C. Quadratic time - $O(n^2)$

D. None of the above

E. I don't know

What is the complexity?

```
def fun_function(n):
    val = 0
    for i in range(n):
        val += 2
        for j in range(1000):
            val += 5
    return val
```

A. Constant time - $O(1)$

B. Linear time - $O(n)$

C. Quadratic time - $O(n^2)$

D. None of the above

E. I don't know

What is the complexity?

```
def fun_function(n):
    val = 0
    for i in range(n):
        val += 2
        for j in range(n):
            val += 1
            for k in range(700):
                val += 5
    return val
```

A. Constant time - $O(1)$

B. Linear time - $O(n)$

C. Quadratic time - $O(n^2)$

D. None of the above

E. I don't know

What is the complexity?

```
def fun_function(n):
    val = 0
    for i in range(5, 30):
        val += 2
        for j in range(3):
            val += 1
            for k in range(700):
                val -= 2
    return val
```

A. Constant time - $O(1)$

B. Linear time - $O(n)$

C. Quadratic time - $O(n^2)$

D. None of the above

E. I don't know

Binary search on sorted list

Algorithm for finding `x` in sorted (ascending) list `L`:

- Pick an index `i` roughly dividing `L` in half
- If `L[i] == x`, return True (if nothing left to search, return False)
- If not:
  - If `L[i] > x`, repeat search on left half of `L`
  - Otherwise repeat search on right half

Find number 24 in a list `L = [?, ?, ?, ?, ?, ?, ?, ?]`

Binary search: first iteration

?	?	?	56	?	?	?	?

`L[i] = 56 > 24` → discard right half and search left half

?	?	?

Binary search: second iteration

?	24	?

`L[i] = 24` → return True

Binary search complexity

Algorithm for finding `x` in list `L`:

- Pick an index `i` roughly dividing `L` in half
- If `L[i] == x`, return True (if nothing left to search, return False)
- If not:
  - If `L[i] > x`, repeat search in left half of `L`
  - Otherwise repeat search in right half

Complexity = # of iterations × constant time per iteration

But **how many** iterations?

- How many times can you split n items in half?
- $\log_2(n)$ (but base of logarithm does not matter for big-O)
- Complexity $O(\log n)$!

Sorting algorithms

So if we have an unsorted list, should we sort it first?

- Suppose complexity $O(sort(n))$
- Is it less work to sort and do a binary search than do a linear search?
- In other words: is $sort(n) + \log(n) < n$?
- No...

In practice: what if we need to search repeatedly, say k times?

- Is $sort(n) + k\log(n) < kn$?
- Depends on k...

Complexity matters

You're planning a trip around the world visiting 10 cities. What's the cheapest route?

Check all alternative routes?

- There are $10 \times 9 \times 8 \times \cdots \times 2 \times 1 = 3\,628\,800$ possible routes
- Factorial complexity $O(n!)$
- Travelling salesperson problem

Review

Measuring algorithm time complexity:

- Number of basic steps taken
- Worst-case analysis
- Focus on large inputs

Searching and sorting (next session) are canonical algorithms problems

Review exercises:

- Big O practice
- Search algorithms